数字素养

版本控制

TAKE YOU ON A TRIP

!!GO!

哈爾濱工業大學(深圳)

HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

探索 从未停止

目录
CONTENT

01 版本控制

02 Git

03 GitHub

数字素养-版本控制

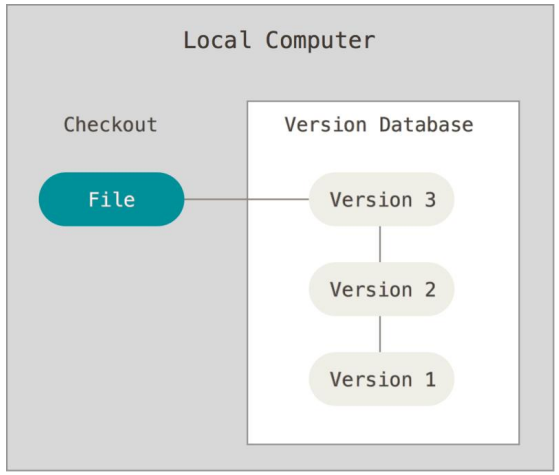
- 参与过2人以上的软件项目开发？合作完成一份调研报告？
- 开发过程中如何解决多个版本备份、回溯、冲突？
- 如果不使用版本控制还会出现什么问题？

  - 备份多个版本，浪费空间和时间
  - 难于恢复到以前的正确版本
  - 容易引发bug
  - 难于解决冲突问题
  - 难于追溯问题代码的修改人和修改时间
  - 版本发布困难

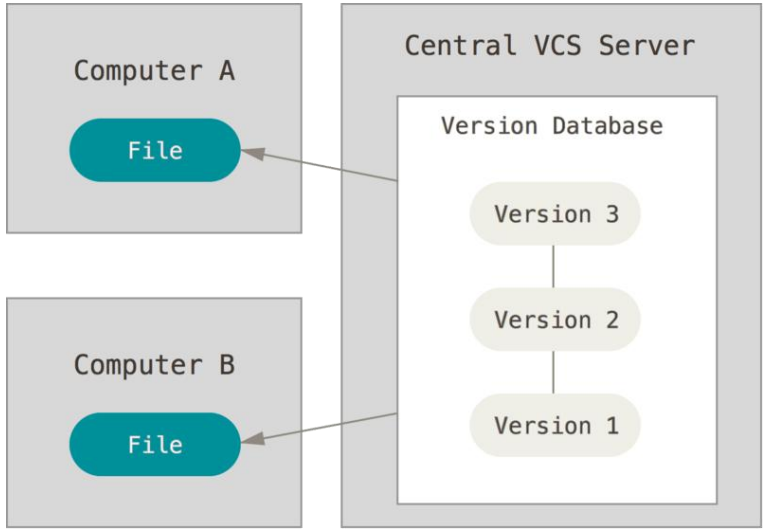
数字素养-版本控制

版本控制（Revision Control）是维护工程蓝图的标准做法，能够跟踪工程蓝图从诞生到正式发布的全过程。是一种记录一个或若干文件内容变化，以便将来查阅特定版本修订情况的系统。

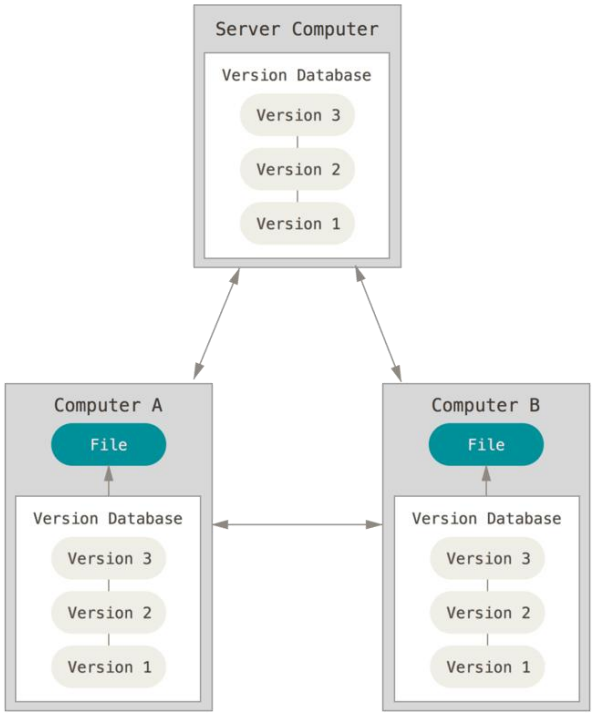
- 本地版本控制系统
- 集中化的版本控制系统
- 分布式版本控制系统



本地版本控制系统



集中化的版本控制系统



分布式版本控制系统

目录
CONTENT

01 版本控制

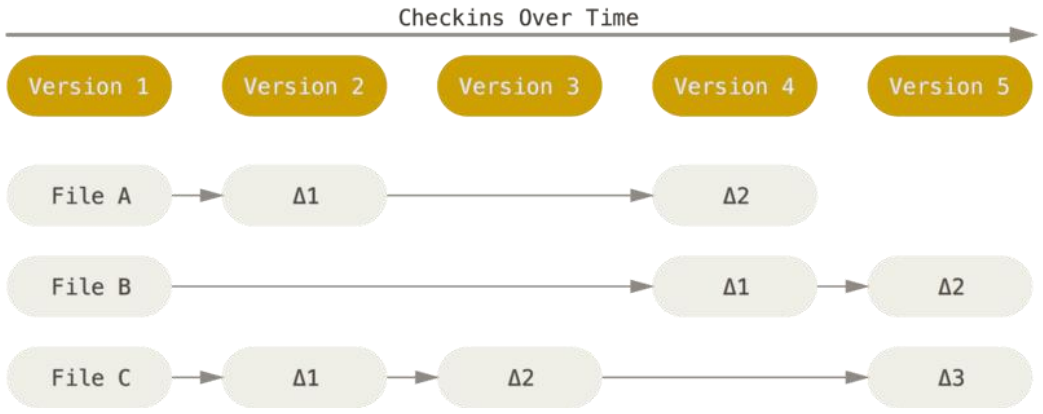
02 Git

03 GitHub

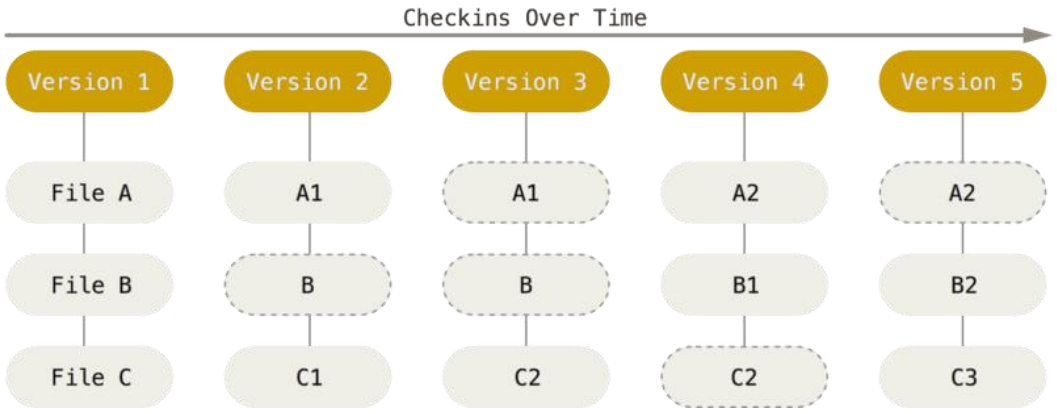
数字素养-Git是什么

■ 直接记录快照，而非差异比较

① 基于差异的版本控制存储的是一组基本文件和每个文件随时间逐步累积的差距。
② Git把数据看做是对小型文件系统的一系列**快照**，如果提交更新，Git会对当时的全部文件创建一个快照，如果文件没有修改，Git不再重新存储该文件，而是只保留一个链接指向之前存储的文件。



基于差异的版本控制



基于快照的Git

Git 不是存“变化”，而是存“状态

数字素养-Git是什么

■ 所有操作都是本地执行

大多数操作只需要访问本地资源和文件。

■ Git保证完整性

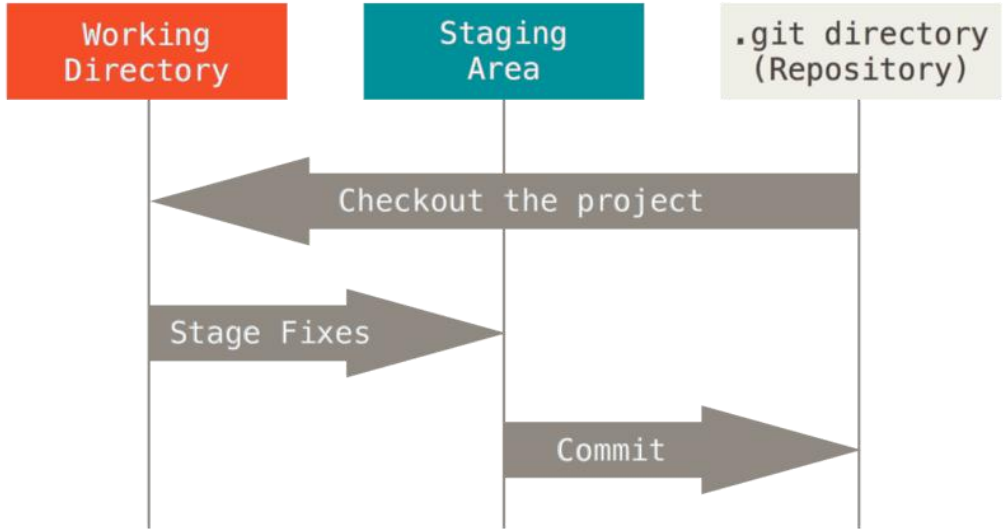
Git所有的数据在存储前都计算校验和，然后以校验和来引用，不可能在Git不知情时更改内容，如在传输过程中丢失信息或损坏内容，Git就能发现。

■ 多数操作仅添加数据

常用的 Git 操作大多仅仅是把数据添加到数据库，一旦提交快照之后就完全不用担心丢失数据。

数字素养-Git基本工作流

■ Git三个阶段



■ 工作区

对项目的某个版本独立提取出来的内容。 这些从 Git 仓库的压缩数据库中提取出来的文件，放在本地磁盘上供你使用或修改。

■ 暂存区

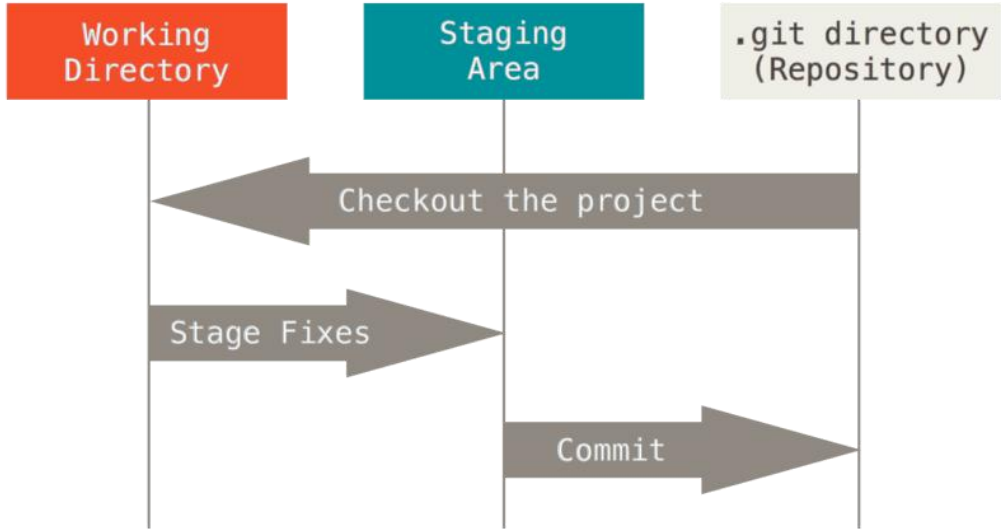
是一个文件，保存了下次将要提交的文件列表信息，一般在 Git 仓库目录中。 按照 Git 的术语叫做“索引”，不过一般说法还是叫“暂存区”。

■ Git 仓库目录

是 Git 用来保存项目的元数据和对象数据库的地方，是 Git 中最重要的部分，从其它计算机克隆仓库时，复制的就是这里的数据。

数字素养-Git基本工作流

■ Git三种状态

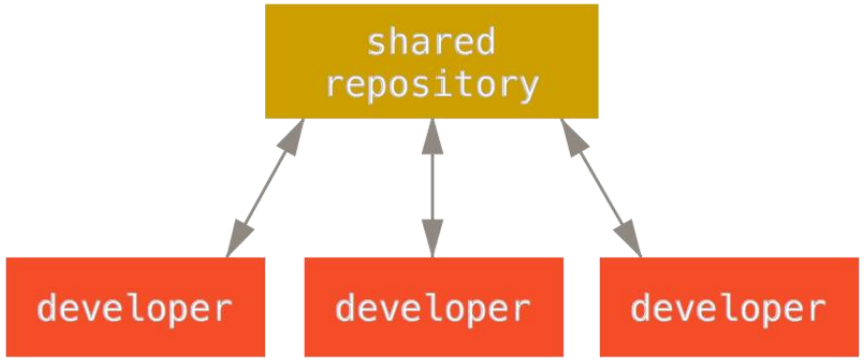


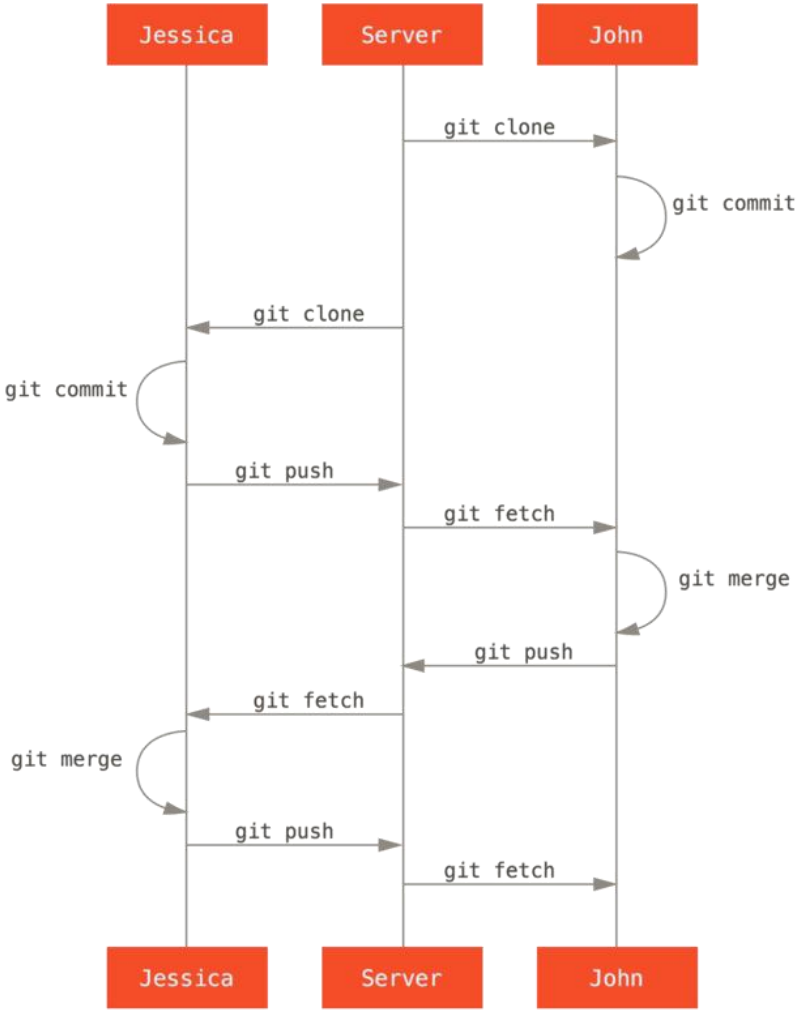
1. 在工作区中修改文件，此时文件已做了修改但还没有放到暂存区，状态是已修改。
2. 将你想要下次提交的更改选择性地暂存，这样只会将更改的部分添加到暂存区，此时状态是已暂存。
3. 提交更新，找到暂存区的文件，将快照永久性存储到 Git 目录，此时文件状态是已提交。

数字素养-Git基本工作流

- **Git协同开发**

安全、有序、可控

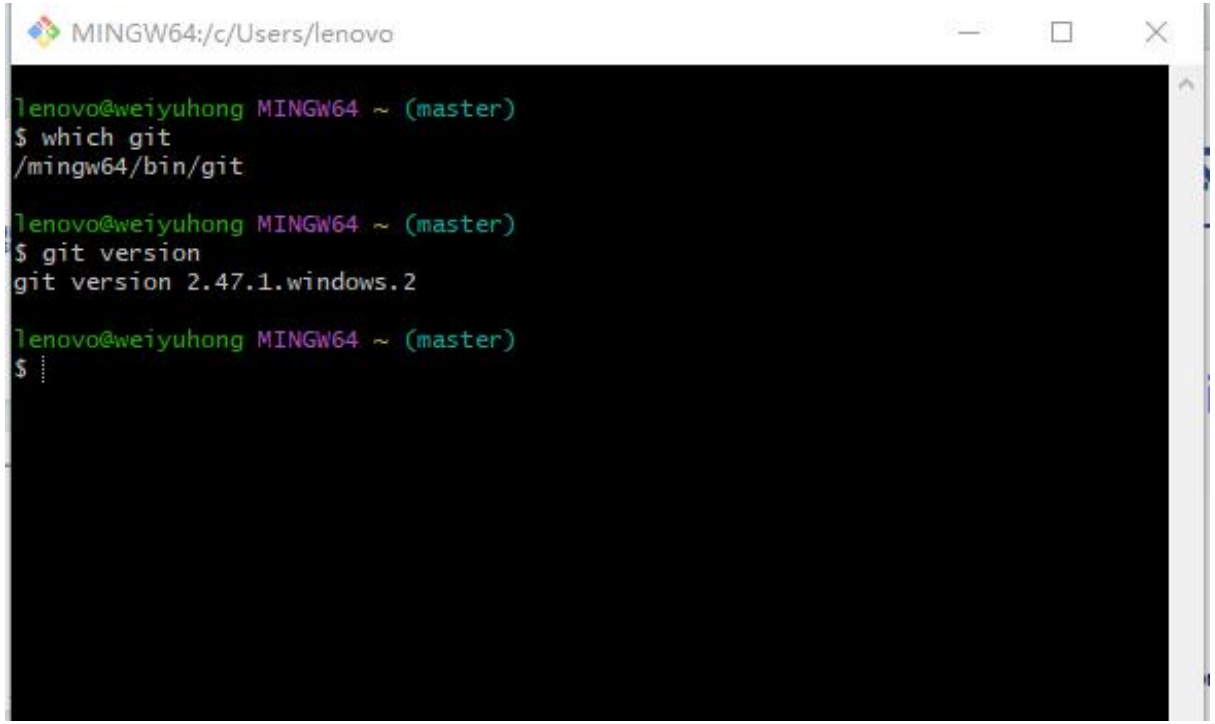




数字素养-Git下载安装

下载安装包https://git-scm.com/download/win

Next



Bash

数字素养-终端机常用命令

bash命令	说明
cd	切换目录
touch	创建文件
mkdir	创建新的目录
ls	列出当前的文件列表
mkdir	创建新的目录
cp	复制文件
mv	文件更名
rm	删除文件
rm *.html	删除所有.html文件

■ **课堂练习1**

① **安装Git**

② **启动Git Bash，练习以下命令**

```
cd /tmp
cd ..
cd d:
ls -al
mkdir demo
cd demo
touch index.html
cp index.html about.html
mv index.html info.html
rm about.html
rm *.html
```

数字素养-Vim编辑器

模式名称	作用	切换方式
正常模式（Normal）	浏览、复制粘贴、删除等	启动 Vim 默认模式
插入模式（Insert）	输入文本	在正常模式下按 i、a、o 等
命令模式（Command）	保存、退出、搜索、替换等	在正常模式下按 :

模式	命令	说明
插入模式（Insert）	i:	Insert
	a:	Append
	o:	在下方新建一行插入
	Esc	退出插入模式，回到正常模式
命令模式（Command）	:q	关闭文件（quit）
	:w	保存（write）
	:wq	**保存并退出**

- **如果不小心进入Vim编辑器，请输入:wq退出，继续其他操作**

数字素养-Git-设置

■ 用户设置

① 使用Git时，首先要设置用户的E-mail信箱和用户名

```
lenovo@weiyuhong MINGW64 /c
$ git config --global user.name "wei"

lenovo@weiyuhong MINGW64 /c
$ git config --global user.email "weiyuhong@hit.edu.cn"
```

② 查看当前设置

```
lenovo@weiyuhong MINGW64 /c
$ git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=openssl
http.sslcainfo=C:/Program Files/Git/mingw64/etc/ssl/certs/ca-bundle.crt
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
user.name=wei
user.email=weiyuhong@hit.edu.cn
```

③ 设置缩写

```
lenovo@weiyuhong MINGW64 /c
$ git config --global alias.co checkout
```

④ 更换编辑器

```
lenovo@weiyuhong MINGW64 /c
$ git config --global core.editor emacs
```

- **--global全局设置，如针对特定项目，--local可以为每个项目设置不同的作者**
- **默认编辑器Vim，可以使用其他编辑器**

数字素养-Git-基本操作

■ 新增、初始Repository

① 创建全新的目录

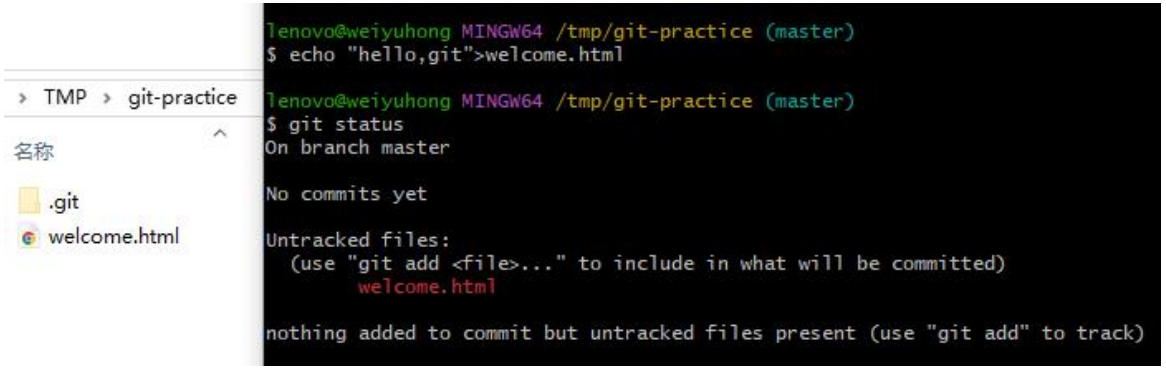
```
lenovo@weiyuhong MINGW64 ~ (master)
$ cd /tmp

lenovo@weiyuhong MINGW64 /tmp
$ mkdir git-practice
c
lenovo@weiyuhong MINGW64 /tmp
$ cd git-practice

lenovo@weiyuhong MINGW64 /tmp/git-practice
$ git init
Initialized empty Git repository in E:/TMP/git-practice/.git/
```

- **mkdir：创建git-practice目录**
- **cd： 切换到git-practice目录**
- **git init：初始化git-practice目录，在git-practice创建一个 .git目录，让Git开始对这个目录进行版本控制。**

② 创建文件

- **除了Git生成的.git隐藏目录，没有其他内容，所以提示“nothing to commit”**

③ 创建内容为“hello，git“的文件welcome.html



- **welcome.html状态为Untracked，表示尚未被加到Git版本控制系统中，还未被“追踪”，只是刚刚加入目录。**

数字素养-Git-基本操作

■ 把文件交给Git暂存区

git add

```
lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        welcome.html

nothing added to commit but untracked files present (use "git add" to track)

lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git add welcome.html
warning: in the working copy of 'welcome.html', LF will be replaced by CRLF t

lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   welcome.html
```

文件已被安置在暂存区（staging area）

- ▸ git add *.html　把.html后缀文件全部加到暂存区
- ▸ git add --all　把全部文件加到暂存区

■ 课堂练习2

① 新增一个文件abc.txt
② 执行git add abc.txt，把文件添加到暂存区
③ 编辑abc.txt
④ 执行git status

```
lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git add abc.txt
warning: in the working copy of 'abc.txt', LF will be replaced by CRLF the next time

lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   abc.txt
        new file:   welcome.html

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   abc.txt
```

abc.txt变成两个文件

- ▸ 对Git而言，修改的内容并没有被再次加入暂存区
- ▸ 确定修改，则需要再次git add abc.txt

数字素养-Git-基本操作

■ 把暂存区内容提交到仓库

git commit

```
lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git commit -m "init commit"
[master (root-commit) 59b2f4a] init commit
 2 files changed, 4 insertions(+)
 create mode 100644 abc.txt
 create mode 100644 welcome.html
```

- -m “init commit”表示这次commit做了什么事，简单清晰
- 到底commit了什么东西?
  Git每次commit只处理暂存区的内容，也就是说，那些没有被加到暂存区的文件，不会被commit到仓库
- 输入的信息很重要!
  commit时如果没有信息，默认是不会完成commit的。它的主要目的是告诉自己和其他人，“这次改动做了什么”
- 一定要有内容才能commit吗?
  加上--allow--empty参数，没有内容也可以commit，但毫无意义

- 为什么要采用二段式?
  一个仓库，仓库门口有个小广场，相当于暂存区。要把存放到仓库的物品，先放到小广场（git add），等货物到达一定数量后，再打开仓库门，把小广场上的货物，送到仓库中（git commit），并记录下这批货物的来源和用途。
  当然可以每来一件货物就打开仓库门存放一次、记录一次，但开仓库门的次数会非常多，导致查阅记录时太过零碎，带来困扰。

- 什么时间可以commit?
  没有标准答案
  ① 完成一个任务时
  ② 下班时
  ③ 想commit时

数字素养-Git-基本操作

■ 查看记录

git log

```
lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ touch index.html
g
lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git add index.html   1 add

lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git commit -m "create index page"   2 commit
[master cedf792] create index page
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 index.html

lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git log
commit cedf7921516b9fe73d91d4a27096e8bd469b7212 (HEAD -> master)
Author: wei <weiyuhong@hit.edu.cn>
Date:   Wed Jun 4 17:25:20 2025 +0800

    create index page

commit 59b2f4af71acc7e26183a936f565a60b2530fbb5
Author: wei <weiyuhong@hit.edu.cn>
Date:   Wed Jun 4 16:49:55 2025 +0800

    init commit
```

越新的信息越显示在上面

作者、时间、提交内容

SHA-1，每个commit ID

- 查找某个人的commit
  git log --online --author= “wei”
- 查找commit信息是否含有某些关键字
  git log --online --grep= “LOL”
- 怎样在commit信息中找到wei
  git log -S “wei”
- 怎样查找某一段时间内的commit
  git log --online --since= “9am” --until “12am” --after=” 2024-01”

数字素养-Git-基本操作

■ 在Git中删除文件

① rm ppmm.html
② git rm ppmm.html

```
lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ rm ppmm.html

lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        deleted:    ppmm.html

no changes added to commit (use "git add" and/or "git commit -a")

lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git add ppmm.html

lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        deleted:    ppmm.html
```

- rm ppmm.html
  add ppmm.html，
- git rm ppmm.html，直接在暂存区，不需要再次add

■ 在Git中变更文件名

① mv index.html world.html
② git mv index.html world.html

```
lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ mv index.html world.html

lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        deleted:    index.html

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        world.html

no changes added to commit (use "git add" and/or "git commit -a")

lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git add --all

lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        renamed:    index.html -> world.html
```

- 与rm类似，请git帮忙改名，少做一步add
- 虽然只是更改文件名，但对Git来讲，是两个动作，一个是删除index.html，一个是新增world.html
- 再次add到暂存区，因为文件内容没有改变，Git判断出只是改名，所以状态改为rename

数字素养-Git-基本操作

■ 挽救已被删除文件

git checkout 1.html

git checkout .

```
lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ ls -al
total 3338
drwxr-xr-x 1 lenovo 197121  0 Jun  4 18:57 ./
drwxr-xr-x 1 lenovo 197121  0 Jun  4 18:51 ../
drwxr-xr-x 1 lenovo 197121  0 Jun  4 18:57 .git/
-rw-r--r-- 1 lenovo 197121  0 Jun  4 18:57 1.html
-rw-r--r-- 1 lenovo 197121  0 Jun  4 18:57 2.html
-rw-r--r-- 1 lenovo 197121  0 Jun  4 18:57 3.html
-rw-r--r-- 1 lenovo 197121 54 Jun  4 16:42 abc.txt
-rw-r--r-- 1 lenovo 197121 11 Jun  4 18:57 welcome.html
-rw-r--r-- 1 lenovo 197121  0 Jun  4 18:57 world.html

lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ rm *.html

lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ ls -al
total 3337
drwxr-xr-x 1 lenovo 197121  0 Jun  4 18:57 ./   已删除所有html文件
drwxr-xr-x 1 lenovo 197121  0 Jun  4 18:51 ../
drwxr-xr-x 1 lenovo 197121  0 Jun  4 18:57 .git/
-rw-r--r-- 1 lenovo 197121 54 Jun  4 16:42 abc.txt

lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git checkout .
Updated 5 paths from the index

lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ ls -al
total 3338
drwxr-xr-x 1 lenovo 197121  0 Jun  4 18:58 ./
drwxr-xr-x 1 lenovo 197121  0 Jun  4 18:51 ../
drwxr-xr-x 1 lenovo 197121  0 Jun  4 18:58 .git/
-rw-r--r-- 1 lenovo 197121  0 Jun  4 18:58 1.html
-rw-r--r-- 1 lenovo 197121  0 Jun  4 18:58 2.html
-rw-r--r-- 1 lenovo 197121  0 Jun  4 18:58 3.html
-rw-r--r-- 1 lenovo 197121 54 Jun  4 16:42 abc.txt
-rw-r--r-- 1 lenovo 197121 11 Jun  4 18:58 welcome.html
-rw-r--r-- 1 lenovo 197121  0 Jun  4 18:58 world.html
```

挽救回来

■ commit后悔，拆掉重做

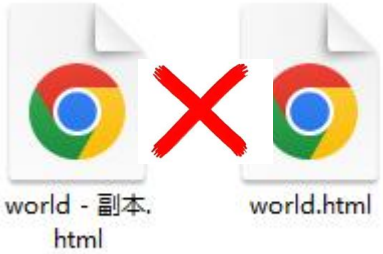
git reset

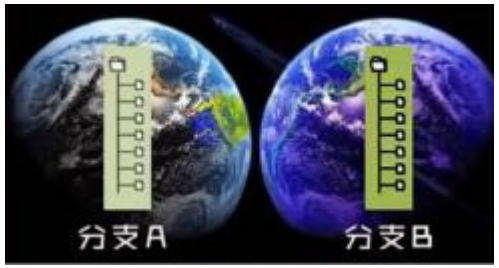
```
lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git log --oneline
9c51919 (HEAD -> master) test for checkout
306b9a0 test test
9a71566 test for rm
cedf792 create index page
59b2f4a init commit

lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git reset 9c51919^
Unstaged changes after reset:      ^表示前一次，相对
D       index.html

lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git log --oneline
306b9a0 (HEAD -> master) test test    已拆掉
9a71566 test for rm
cedf792 create index page
59b2f4a init commit

lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git reset cedf792
Unstaged changes after reset:      清楚写明退回到哪个commit
D       index.html

lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git log --oneline
cedf792 (HEAD -> master) create index page
59b2f4a init commit
```

数字素养-Git-分支

■ **什么是分支**

可以理解为当前工作目录的一个副本

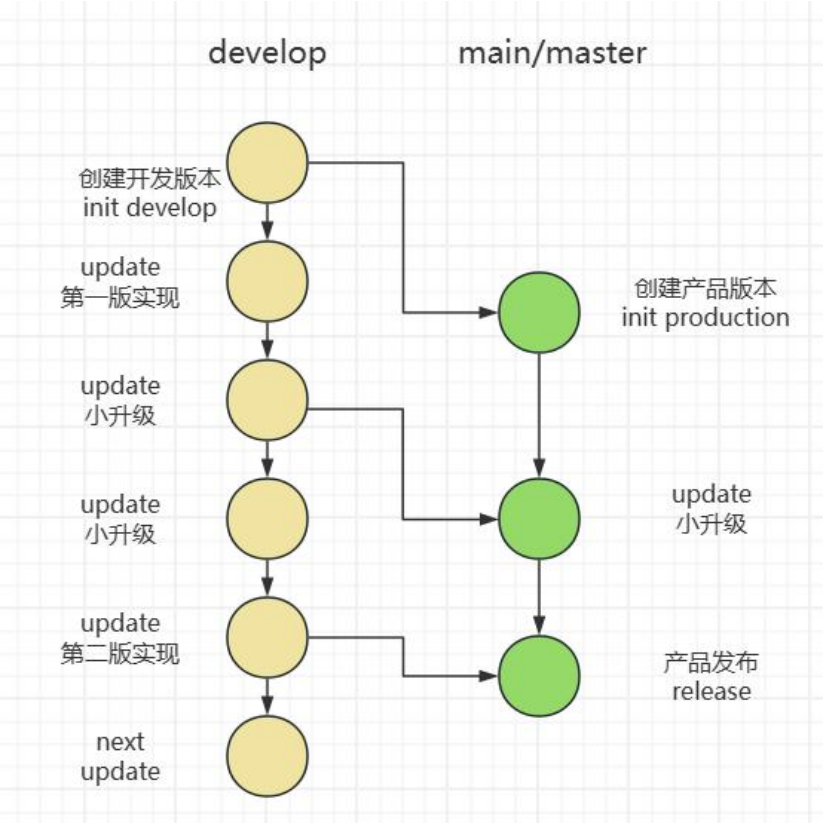




当前分支

■ **为什么要使用分支**

将项目划分为多条线：
开发分支进行修改（开发线）
稳定后合并到**主线**（产品线），保证主线稳定性



■ **主分支（main/master）**
▸ 包含所有最终修改的历史，项目最终版本
■ **开发分支（develop）**
▸ 与主分支平行，当开发分支到达稳定状态并准备发布时，与主分支合并

数字素养-版本控制

增量开发，是软件工程中一种常用的软件开发过程思想。指在软件开发过程中，先开发主要功能模块，再开发次要功能模块，逐步完善，最终开发出符合需求的软件产品。

好处：

1. 从一个能运行的程序开始，每次只增加少量改动。能够清楚定位错误的源头，减少调试时间。
2. 应对软件产品需求的不确定因素，确定增量发布版本的过程，也是软件项目风险控制的过程。

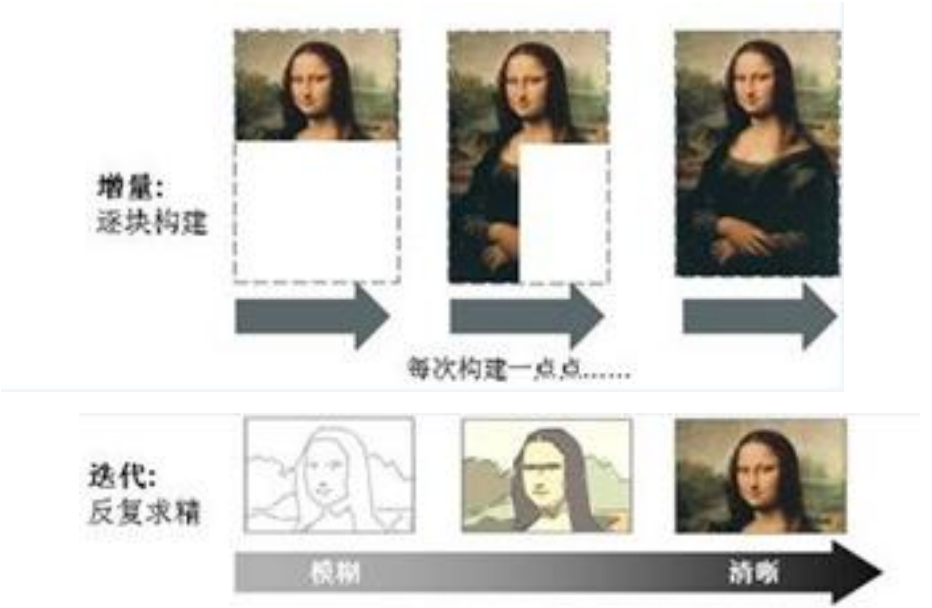
版本控制：开发过程中的“记录器”

VS

增量开发：“搭积木”，一步步地添加新功能

VS

迭代开发：“打磨作品”，对已有系统不断优化、完善



数字素养-Git-分支

■ **新增分支**

git branch

git branch cat

```
lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git branch
* master
```

不加参数，表示当前项目有哪些分支

```
lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git branch cat
```

增加一个分支cat

```
lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git branch
  cat
* master
```

*表示当前分支

■ **更改分支名称**

git branch -m cat tiger

```
lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git branch -m cat tiger

lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git branch
  dog
* master
  tiger

lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git branch -m master slave
```

master分支

```
lenovo@weiyuhong MINGW64 /tmp/git-practice (slave)
$ git branch
  dog
* slave
  tiger
```

■ **删除分支**

git branch -d tiger

git branch -D dog

```
lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git branch
  dog
* master
  tiger

lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git branch -d tiger
Deleted branch tiger (was cedf792).

lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git branch
  dog
* master

lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git branch -d dog
Deleted branch dog (was cedf792).

lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git branch
* master
```

- *表示当前分支
- 没有什么分支是不能删除的，包括master
- 只有”当前所在的分支“不能删，但可以先切换到其他分支，然后再删除
- 如果要删除的分支没有被完全合并，Git会提示
- -D表示强制删除

数字素养-Git-分支

■ **切换分支**

git checkout cat

- 要切换到哪个分支，要先有那个分支

```
lenovo@weiyuhong MINGW64 /tmp/git-practice (cat)
$ touch cat1.html

lenovo@weiyuhong MINGW64 /tmp/git-practice (cat)
$ git add cat1.html
g
lenovo@weiyuhong MINGW64 /tmp/git-practice (cat)
$ git commit -m "add cat 1"
[cat 40ce450] add cat 1
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 cat1.html

lenovo@weiyuhong MINGW64 /tmp/git-practice (cat)
$ touch cat2.html

lenovo@weiyuhong MINGW64 /tmp/git-practice (cat)
$ git add cat2.html

lenovo@weiyuhong MINGW64 /tmp/git-practice (cat)
$ git commit -m "add cat 2"
[cat c377a53] add cat 2
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 cat2.html
```

cat分支新增两个文件

```
lenovo@weiyuhong MINGW64 /tmp/git-practice (cat)
$ ls -al
total 3338
drwxr-xr-x 1 lenovo 197121  0 Jun  4 20:06 ./
drwxr-xr-x 1 lenovo 197121  0 Jun  4 20:06 ../
drwxr-xr-x 1 lenovo 197121  0 Jun  4 20:06 .git/
-rw-r--r-- 1 lenovo 197121  0 Jun  4 18:58 1.html
-rw-r--r-- 1 lenovo 197121  0 Jun  4 18:58 2.html
-rw-r--r-- 1 lenovo 197121  0 Jun  4 18:58 3.html
-rw-r--r-- 1 lenovo 197121 54 Jun  4 16:42 abc.txt
-rw-r--r-- 1 lenovo 197121  0 Jun  4 20:05 cat1.html
-rw-r--r-- 1 lenovo 197121  0 Jun  4 20:06 cat2.html
-rw-r--r-- 1 lenovo 197121 11 Jun  4 18:58 welcome.html
-rw-r--r-- 1 lenovo 197121  0 Jun  4 18:58 world.html

lenovo@weiyuhong MINGW64 /tmp/git-practice (cat)
$ git checkout master
D       index.html
Switched to branch 'master'
```

切换到master分支

切换到master分支

```
lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ ls -al
total 3338
drwxr-xr-x 1 lenovo 197121  0 Jun  4 20:08 ./
drwxr-xr-x 1 lenovo 197121  0 Jun  4 20:06 ../
drwxr-xr-x 1 lenovo 197121  0 Jun  4 20:08 .git/
-rw-r--r-- 1 lenovo 197121  0 Jun  4 18:58 1.html
-rw-r--r-- 1 lenovo 197121  0 Jun  4 18:58 2.html
-rw-r--r-- 1 lenovo 197121  0 Jun  4 18:58 3.html
-rw-r--r-- 1 lenovo 197121 54 Jun  4 16:42 abc.txt
-rw-r--r-- 1 lenovo 197121 11 Jun  4 18:58 welcome.html
-rw-r--r-- 1 lenovo 197121  0 Jun  4 18:58 world.html
```

无cat1/2

master分支ls无cat1和cat2

数字素养-Git-分支

■ **合并分支**

git merge cat

```
lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git merge cat
Updating cedf792..c377a53
Fast-forward
 cat1.html | 0
 cat2.html | 0
 2 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 cat1.html
 create mode 100644 cat2.html

lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ ls -al
total 3338
drwxr-xr-x 1 lenovo 197121  0 Jun  4 20:28 ./
drwxr-xr-x 1 lenovo 197121  0 Jun  4 20:25 ../
drwxr-xr-x 1 lenovo 197121  0 Jun  4 20:28 .git/
-rw-r--r-- 1 lenovo 197121  0 Jun  4 18:58 1.html
-rw-r--r-- 1 lenovo 197121  0 Jun  4 18:58 2.html
-rw-r--r-- 1 lenovo 197121  0 Jun  4 18:58 3.html
-rw-r--r-- 1 lenovo 197121 54 Jun  4 16:42 abc.txt
-rw-r--r-- 1 lenovo 197121  0 Jun  4 20:28 cat1.html
-rw-r--r-- 1 lenovo 197121  0 Jun  4 20:28 cat2.html
-rw-r--r-- 1 lenovo 197121 11 Jun  4 18:58 welcome.html
-rw-r--r-- 1 lenovo 197121  0 Jun  4 18:58 world.html
```

■ **课堂练习3-分支练习**

① 新建分支
② 修改名称
③ 删除分支
④ 切换分支
⑤ 合并分支

目录
CONTENT

01 版本控制

02 Git

03 GitHub

数字素养-Git第三方托管

- Git进行协同开发**需要建立服务器**;
- 使用托管网站快速建立并开始项目，无需进行服务器维护和监控工作;
- 将开源代码托管在公共托管网站，减少维护成本，代码分享和协同开发平台!

Public Git hosting sites

Here are some places that provide free Git hosting. Check on GitServer if you want to host your own repository.

List is limited to sites that provide explicit Git hosting, not including generic hosting sites that can be used to host Git repositories.

Provider	Framework is open-source?	Support for other SCM	Open-source repositories	Space (GB)	Free private repositories
Beanstalk	No	SVN	No	0.1	1 projects, 1 user
bitbucket.org	No	No	Yes	∞	∞ projects, 5 collaborators
Codebase	No	Mercurial/SVN	Public access available	0.05	1 project (unlimited repos), 2 collaborators
Codeberg	Yes	No	Yes	∞ (reasonable use)	∞ for certain use cases only
Helix TeamHub	No	Mercurial/SVN	No	1	∞ projects, 5 users
GitHub	No	SVN	Yes	∞	∞ projects, 3 collaborators
GitLab.com	partial (see description below)	No	Yes	∞	∞ projects, ∞ collaborators
ProjectLocker	No	SVN	Read-only http	0.05	1 project, 1 collaborators
repo.or.cz	Yes	No	Yes	0.4	No
RocketGit	Yes	No	Yes	∞	∞
SourceForge.net	Yes	Hg, SVN	Yes	∞	No
SourceHut	Yes	Hg	Yes	∞	∞ during public alpha
Unfuddle	No	SVN	Yes	0.2	1 project, 2 collaborators
Visual Studio Team Services	No	TFVC	No	∞	∞, 5 users

- **三步骤**
  - ① **创建本地库**
  - ② **创建远程库**
  - ③ **将两者关联**

数字素养-远端协作GitHub

■ 在GitHub上创建新项目

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? Import a repository.

Required fields are marked with an asterisk (*).

Owner * Repository name *

Beryl-Wei99 / git-practice

git-practice is available.

Great repository names are short and memorable. Need inspiration? How about studious-carnival ?

Description (optional)

wei's example

Public
Anyone on the internet can see this repository. You choose who can commit.

Private
You choose who can see and commit to this repository.

Initialize this repository with:

Add a README file
This is where you can write a long description for your project. Learn more about READMEs.

Add .gitignore

.gitignore template: None

Choose which files not to track from a list of templates. Learn more about ignoring files.

Choose a license

License: None

A license tells others what they can and can't do with your code. Learn more about licenses.

You are creating a public repository in your personal account.

Create repository

git-practice Public

Pin Watch 0

Set up GitHub Copilot
Use GitHub's AI pair programmer to autocomplete suggestions as you code.
Get started with GitHub Copilot

Add collaborators to this repository
Search for people using their GitHub username or
Invite collaborators

Quick setup — if you've done this kind of thing before

Set up in Desktop or HTTPS SSH git@github.com:Beryl-Wei99/git-practice.git

Get started by creating a new file or uploading an existing file. We recommend every repository include a README, LICENSE, and .gitignore.

...or create a new repository on the command line

```
echo "# git-practice" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin git@github.com:Beryl-Wei99/git-practice.git
git push -u origin main
```

...or push an existing repository from the command line

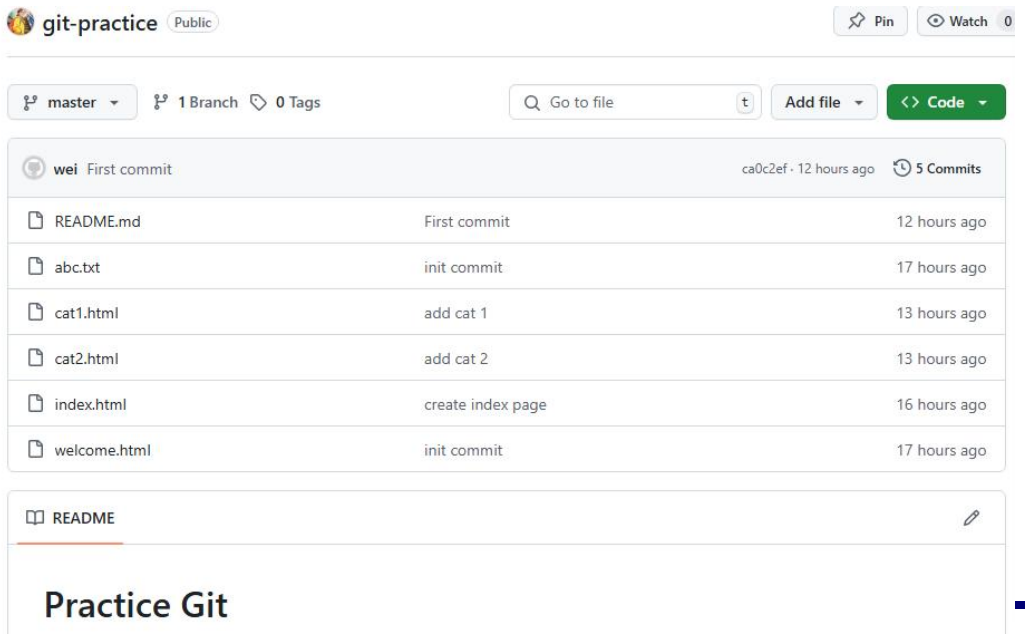
```
git remote add origin git@github.com:Beryl-Wei99/git-practice.git
git branch -M main
git push -u origin main
```

GitHub-Build software better, together

数字素养-远端协作GitHub

```
lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ echo "# Practice Git" > README.md

lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git init
Reinitialized existing Git repository in E:/TMP/git-practice/.git/

lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git add README.md
warning: in the working copy of 'README.md', LF will be replaced by CRLF the

lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git commit -m "First commit"
[master ca0c2ef] First commit
 1 file changed, 1 insertion(+)
 create mode 100644 README.md

lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git remote add origin git@github.com:Beryl-wei99/git-practice.git

lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git push -u origin master
Enumerating objects: 14, done.
Counting objects: 100% (14/14), done.
Delta compression using up to 8 threads
Compressing objects: 100% (11/11), done.
Writing objects: 100% (14/14), 1.10 KiB | 376.00 KiB/s, done.
Total 14 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), done.
remote: This repository moved. Please use the new location:
remote:   git@github.com:Beryl-Wei99/git-practice.git
To github.com:Beryl-wei99/git-practice.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.
```

- **$ git remote add origin git@github.com:Beryl-wei99/git-practice.git**
  - ① git remote 与远端相关操作
  - ② add 要加入一个远端的节点
  - ③ origin 代名词，指后面一串GitHub服务器的位置，远端节点默认名称是origin。如果是从服务器上clone下来的，默认名称就是origin。
- **$ git push -u origin master**
  - ① 把master分支推向origin位置
  - ② 在origin远端服务器上，如果msster不存在，则创建一个master分支
  - ③ 如果存在master分支，就会移动服务器master分支的位置，使它指向当前最新进度



数字素养-开始使用Git

- **去线上抓取内容**

  git fetch

  git merge

```
lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git fetch
```

无返回信息

```
lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git fetch
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 968 bytes | 10.00 KiB/s, done.
From github.com:Beryl-wei99/git-practice
   ca0c2ef..435032f  master     -> origin/master
```

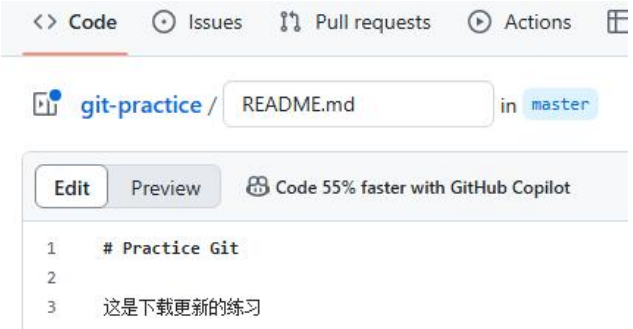
修改README文件后，再次fetch

```
lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git merge origin/master
Updating ca0c2ef..435032f
Fast-forward
 README.md | 2 ++
 1 file changed, 2 insertions(+)
```

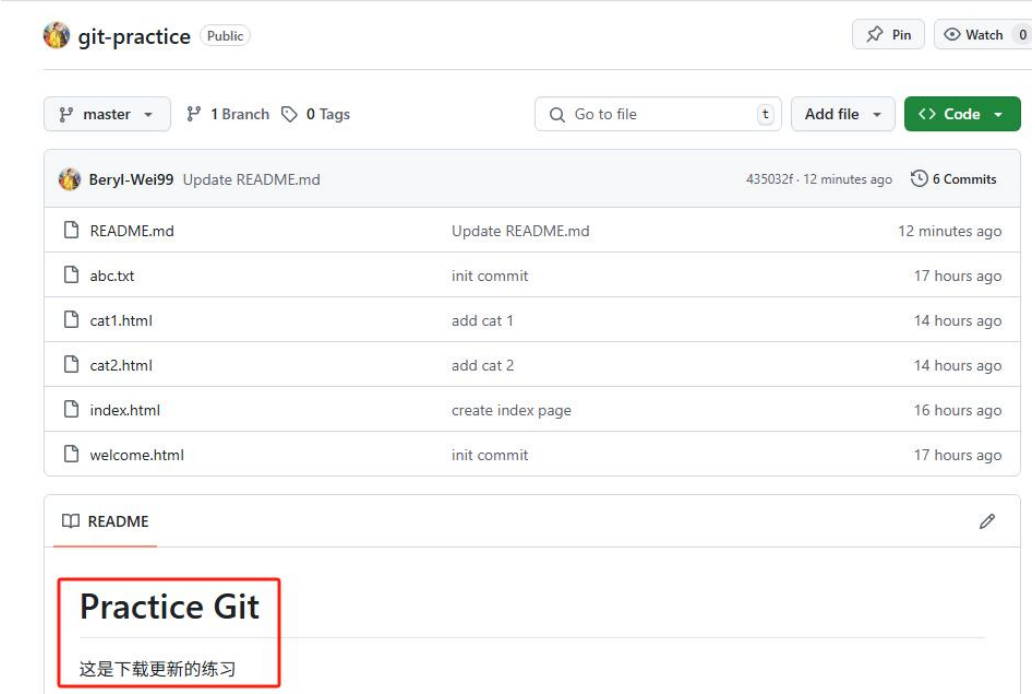
origin领先master，合并

- **git pull　去线上抓取内容，并且更新本机进度**

  git pull=git fetch+git merge



编辑README文件



数字素养-开始使用Git

■ SSH从服务器上取得Repository

git clone

```
lenovo@weiyuhong MINGW64 /tmp/git-practice (master)
$ git clone git@github.com:Beryl-wei99/git-practice.git ~/Desktop/bbb
Cloning into 'C:/Users/lenovo/Desktop/bbb'...
remote: Enumerating objects: 17, done.
remote: Counting objects: 100% (17/17), done.
remote: Compressing objects: 100% (10/10), done.
remote: Total 17 (delta 4), reused 13 (delta 3), pack-reused 0 (from 0)
Receiving objects: 100% (17/17), done.
Resolving deltas: 100% (4/4), done.
```

■ $ git clone git@github.com:Beryl-wei99/git-practice.git ~/Desktop/bbb

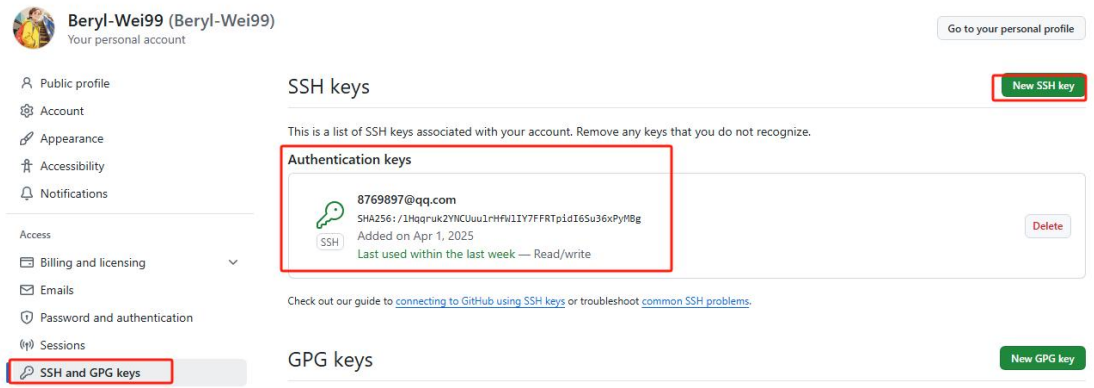
① Beryl-wei99 账户名
② git-practice.git 项目名
③ ~/Desktop/bbb 指定clone路径

■ clone和pull的区别

- clone：第一次看到某个项目，想要下载到自己的计算机中
- pull：已经下载的项目，想要更新到最新的版本

■ SSH key

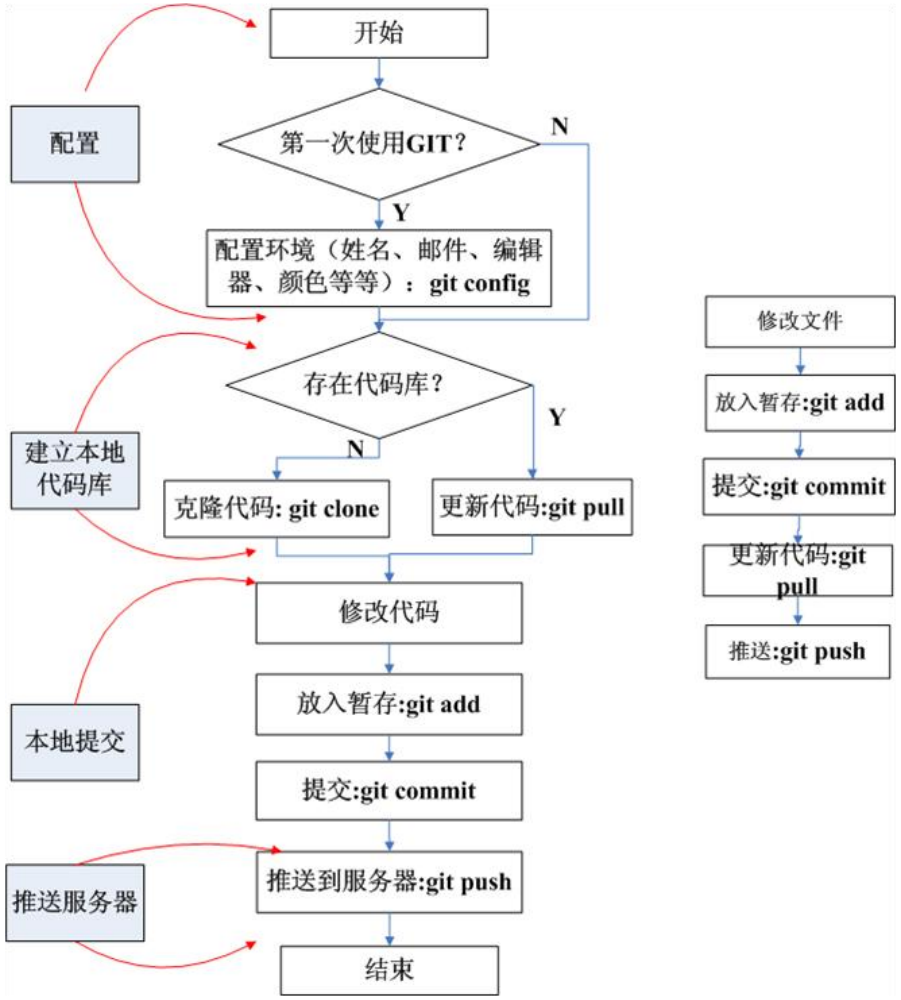
SSH 就像是一把“数字钥匙”，让你安全地远程“开门”到另一台电脑，安全地传输数据。



■ 在 GitHub 中使用 SSH：

- 避免每次 push/pull 输入用户名和密码
- 实现更加安全的身份验证
- 支持脚本和自动化操作时免交互登录

数字素养-Git操作流程图



Git操作流程图

■ **课堂练习4-GitHub**

① **注册登录，https://github.com/**
② **请借助AI工具，配置SSH连接到GitHub**
③ **创建本地库，推送到GitHub**
④ **从远程仓库克隆到本地**

Git和GitHub的详细使用请参考：

https://git-scm.com/book/zh/v2

谢谢！

哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

GO!
TAKE YOU ON A TRIP

探索 从未停止